Week 2 Notes TLS

Guiding Questions: What do websites use to keep data safe (TLS)? How does the TLS handshake keep messages safe between the client and server? How does it guarantee future session safety if a previous session is compromised?

Overall Objective: How does a client trust who they're exchanging messages with over the internet?

Weekly Goal Sentence (keep this visible the entire hour): By the end of this week I should be able to explain how a client connects to a server securely and why that connection can be trusted to my family.

Monday 05/11 Notes

Goal: Understand what problem TLS solves and how it sits on top of the existing encryption knowledge from Week 1.

Link 1: https://www.ibm.com/think/topics/transport-layer-security

- TLS full definition: A cryptographic protocol that helps secure communications over unprotected computer networks such as the Internet
- TLS provides end-to-end authentication, confidentiality, and data integrity
- Applications include:
    - Emails
    - Messaging
    - Voice over IP (VoIP)
    - Virtual Private Networks (VPNs)

- Foundation of Hypertext Transfer Protocol Secure (HTTPS): Application layer protocol that enables encrypted data exchanges between web applications and most major web browsers
- Two layers: TLS handshake protocol and record protocol
- Handshake protocol authenticates web server and client
- The client is defined as the device connecting to the server.
- The record protocol verifies and secures the data for transport
- These TLS layers sit above a transport protocol such as the Transmission Control Protocol/Internet Protocol (TCP/IP) model. Considered “the glue that holds the Internet together”
- TLS is the successor to the Secure Sockets Layer (SSL) protocol formally standardized in 1996 by the Internet Engineering Task Force (IETF)
- In January 1999, the IETF changes the name of SSL 3.0 to TLS
- TLS is critical to protecting online communications from unauthorized access. Safeguarding sensitive data is essential for protection of Personally Identifiable Information (PII) freely exchanged on the internet.
- TLS need to protect against:
    - Hackers
    - Tampering
    - Eavesdropping
    - Other cybersecurity threats such as data breaches, malware, or man-in-the-middle attacks
- The HTTPS padlock icon that’s found on most browsers ensures that the website is trustworthy and safe. It also means that the website has a valid TLS certificate
- A certificate is a digital credential issued by a certificate authority (CA) which verifies a website’s identity and enables encrypted connection.
- TLS ensures secure Internet communication by enforcing three core properties:
    - Authentication: Verifies the identities of the sender and receiver, and the origin and destination of the data.
    - Confidentiality: Ensures data can only be accessed by its intended recipient
    - Integrity: Ensures data cannot be modified in storage or transmission without changes being detected
- A key is a random string of numbers or letters.
- When used with a cryptographic algorithm it encrypts or decrypts information.
- The algorithms used for data transferred over a network are either secret key cryptography or public key cryptography
- Secret key cryptography or symmetric encryption uses one shared key between two parties
- Most sensitive data sent in a TLS session is sent using secret-key cryptography.
- TLS uses cryptographic hash functions to help ensure privacy and data integrity
- Hash function: Generate a fixed-size hash value from input data. Serves as a digital footprint
- Compared before and after transmission to determine if data sent was tampered with.

- Message Authentication Code (MAC) is like a cryptographic hash, but it also authenticates the sender. Applies a secret key to hashed messages creating whats known as a hashed message authentication code (HMAC)
- Public Key Cryptography or asymmetric cryptography uses a linked pair of keys. One public to encrypt and one private to decrypt data.
- These capabilities help establish trust when integrated with public key infrastructure (PKI).
- Public Key Certificates (AKA digital certificates or SSL/TLS certificates) bind public keys to verified identities through a certificate authority.
- Certificates also provide the basis for digital signatures used to authenticate a client or web server.
- Digital Signature: Once a hash is created for a message, the hash is encrypted with the sender's private key.
- The digital signature can be verified by anyone using the sender's public key, providing validation of the sender's identity and data's integrity.
- TLS handshake protocol establishes a secure connection between the client and the web server.
    - Public Key cryptography is used to authenticate the server (and sometimes the client) using a digital certificate.
    - The client and server then agree on a cipher suite (a set of encryption algorithms) and perform a key exchange to securely establish shared session keys (secret keys for encryption and hashing)
    - Once the session keys are established, the secure session is set up.
    - All subsequent data transmitted is encrypted and authenticated using secret key cryptography
- The TLS record protocol is responsible for securing data transmitted during the TLS connection.
- Uses the cipher suite and keys negotiated during the handshake.
    - Cipher suite defines which algorithms should be used for key exchange, encryption, and message authentication.
    - The secret keys are used to encrypt outgoing data and decrypt incoming data.
    - Keys are also used to create MACs
- Through these mechanisms, the record protocol ensures the authentication, integrity, and confidentiality of the connection.
- The TLS handshake takes around 200 to 300 milliseconds to complete. Steps for a TLS 1.3 handshake:
    - Client hello: Client sends ClientHello message with the supported TLS version, cipher suites it supports, random byte string (client_random), and an ephemeral key share using the Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchange method
    - Server hello and key exchange: Server responds with a ServerHello message containing the selected cipher suite, another random byte string (server_random), and its own ephemeral key share. This establishes the key exchange parameters

        - Both parties can compute a shared secret using the ECDHE key exchange method
        - The shared secret is used to derive the handshake keys.
        - Server then sends its digital certificate, a CertificateVerify message (signed with its private key for authentication), and a Finished message (encrypted with the handshake key)
        - If the server requires client authentication, it sends a CertificateRequest message. The client then sends its certificate and a CertificateVerify message.
    - Authentication and Finished: Client verifies the server's certificate is issued by a trusted certificate authority, is valid and not revoked, and that the domain matches.
        - Also verifies the server's CertificateVerify message using the public key from the certificate and the Finished message using the handshake key.
        - Client then sends its own Finished message using the handshake key, and the handshake is confirmed complete.
        - Both parties can now exchange messages using symmetric encryption
- TLS can be implemented using several key exchange methods. Some include: (I only included this section in my notetaking because the latest TLS version only uses ECDHE)
    - Diffie-Hellman with variations:
        - Diffie-Hellman Ephemeral (DHE): Ephemeral means a temporary or single-use key is generated for each session. This provides forward secrecy, ensuring that the compromise of long-term keys does not compromise past sessions.
        - Elliptic Curve Diffie-Hellman (ECDH)
        - Elliptic Curve Diffie-Hellman Ephemeral (ECDHE): mandated key exchange method for TLS 1.3
    - RSA: Not supported in TLS 1.3 because it does not provide forward secrecy, but can still be used for authentication
    - Pre-shared key (PSK): recommended to be used along with ECDHE to provide forward secrecy
- TLS 1.3 was established in August 2018. Can be implemented with backward-compatibility, but not directly compatible with previous TLS versions.

It took 45 minutes to go through this article.

Initial Summary:

TLS solves the problem of authenticating and securing communications across the internet. Securing communications is done by one of the TLS protocols, the handshake protocol. In this protocol, public key cryptography is used to establish connections between the client and server. The client sends a message with its ciphersuites, public key (what I believe IBM meant by "random string"), an ephemeral symmetric key encrypted with the client's private key, and a ClientHello message. The server responds by sending its digital certificate, another public key,

the agreed upon ciphersuite, and a MAC signed by its private key. The server decrypts the ephemeral symmetric key to be used to establish the exchange of messages and a Finished message (the rest I don't remember for this step). The client verifies the certificate, the MAC of the server to ensure authentication of the server, and the client and server can now communicate with the ephemeral symmetric key they shared with each other.

One Sentence Answer to Goal:
TLS solves the problem of authenticating and securing communications across the internet by connecting clients and servers together using asymmetric cryptography to establish the connection and shared symmetric cryptography key to exchange messages between each other.

Tuesday 05/12 Notes

Summary from memory (10 min timer):

[write before reading Monday notes]

Fuzzy or missing:

[5 min timer]

Research focus today:

[gaps to fill]

[notes here]

Wednesday 05/13 Notes

Consolidation only — no new sources.
Read all notes. Answer this out loud:
[weekly goal question]

Gaps identified:

[anything still broken]

[notes here]

Thursday 05/14

Talk-through first — then build slides.

Narrative spine:

[write the story in 3-5 sentences before opening slides]

Week 2 Retrospective

What I did well:

What I did badly:

What to fix Week 3:

Friday 05/15 Presentation Remarks:

[family feedback here]